# Advanced Data Management
# Quiz Refresher: Chapter 4, Questions

**Query processing and operator algorithms: the iterator model, external sort, the three joins, aggregation and reading PostgreSQL plans; answers in the separate answer sheet**

Advanced Data Management

2026-10-27

## How to use this document

Three rounds, answers in the separate answer sheet. Round A is 20 one-line recall questions. Round B is eight multiple-choice questions in the mock's format, letter plus one-line justification. Round C is five problems in the style of the mock's Parts 2 and 3; every number is new, so the chapter's worked examples will not carry you, the formulas must. Budget 20 minutes for A and B, 60 minutes for C. Mock questions 1.1, 1.5, 2.4 and 3.4 live in this chapter, and C2 and C3 are built to look like them.

## Round A: rapid recall

A1. What is a pipeline breaker? Name three operators that are.
A2. Which of these is *not* a pipeline breaker: Sort, hash build, HashAggregate, GroupAggregate over sorted input?
A3. What do selection and projection cost in page I/O, and why?
A4. "To find equal things, either ... or ...". Complete it, and name three operators built on it.
A5. External sort: formula for the number of initial runs, and the merge fan-in. Why not $M$?
A6. Formula for the number of passes. What is the "1"?
A7. Formula for the total I/O of an external sort.
A8. What PostgreSQL setting plays the role of $M$? Give the two `Sort Method` lines you can see in a plan.
A9. Cost of the simple nested-loop join and of the block nested-loop join. What are the two reserved buffers in $M - 2$?
A10. Cost of the index nested-loop join.
A11. Which input goes on the outside of a nested loop, and why?
A12. Cost of a sort-merge join, and the two situations where it wins.

A13. Hash join: which side is built, and what is the cost when the build fits?

A14. When does Grace hash join happen, what is its cost, and which plan line reveals it?

A15. Which join algorithm handles an inequality condition? Which cannot, and why?

A16. In `actual time=a..b`, what are $a$ and $b$? What does $a = b$ on a Hash node tell you?

A17. `loops=20000` under an inner Index Scan means what?

A18. HashAggregate versus GroupAggregate: what decides which one the planner picks?

A19. What does `SET enable_hashjoin = off` do, and what must you run afterwards?

A20. Why does a hash join give no ordering guarantee on its output?

## Round B: multiple choice, letter plus justification

Each question is 2 points: 1 for the letter, 1 for a one-line justification.

B1. **Sort passes.** $B = 5{,}000$ pages, $M = 51$ buffers. Passes, counting run generation?
(a) 2 (b) 3 (c) 4 (d) 99

B2. **Two-pass sort.** Smallest $M$ that sorts $B = 4{,}000$ pages in exactly two passes?
(a) 63 (b) 64 (c) 65 (d) 2,000

B3. **Block nested loop.** $B_R = 500$, $B_S = 4{,}000$, $M = 27$, $R$ outer. Page reads?
(a) 4,500 (b) 76,500 (c) 80,500 (d) 2,000,500

B4. **Not a breaker.** Which operator can emit output before consuming all of its input?
(a) Sort (b) Hash (build) (c) HashAggregate (d) GroupAggregate over sorted input

B5. **Grace.** Hash join with build side $B_R = 800$, probe side $B_S = 20{,}000$; the build does not fit in memory. Approximate cost?
(a) 20,800 (b) 41,600 (c) 62,400 (d) 16,000,800

B6. **Batches.** A plan shows `Hash ... Batches: 8`. This means:
(a) the hash table has 8 buckets (b) the build side exceeded `work_mem` and was partitioned to disk in 8 pieces (c) the sort took 8 passes (d) the probe side was scanned 8 times

B7. **No hash.** Which join condition rules out a hash join?
(a) `a.id = b.id` (b) `a.x = b.y AND a.z = b.w` (c) `a.start_ts < b.ts` (d) `a.id = b.id` with no index on either side

B8. **Sort then group.** The planner chose `Sort` then `GroupAggregate` over `HashAggregate` for a `GROUP BY`. The most likely reason:
(a) the input is tiny (b) there are so many groups the hash table would

exceed `work_mem` (c) there are very few groups (d) `SUM` always needs sorted input

## Round C: midterm-style problems

**Problem: C1. External sort at scale** [8 pts]

A table `clicks` occupies $B = 600{,}000$ pages. A query must sort it by `ts`; `work_mem` gives $M = 500$ buffer pages.

(a) Number of initial runs, merge fan-in, number of passes, total page I/O.

(b) The smallest $M$ that sorts the table in two passes. Show the inequality you solve.

(c) Translate that $M$ into a `work_mem` setting in MB (8 KB pages).

(d) With $M = 600{,}000$ what happens, and what `Sort Method` line would the plan show?

(e) Why can production systems not simply set `work_mem` to 1 GB for everyone?

**Problem: C2. Five ways to join** [12 pts]

`sessions`: $B_R = 2{,}000$ pages, $|R| = 100{,}000$ rows (50 per page). `clicks`: $B_S = 80{,}000$ pages. The join is `s.id = c.session_id`. $M = 42$ buffer pages. `clicks` has a B+ tree on `session_id` of height 3, and every session has 40 clicks; assume each matching click costs one heap page read (worst case).

(a) Simple nested loop, `sessions` outer. Cost? Why is this number absurd?

(b) Block nested loop, `sessions` outer, then `clicks` outer. Which is better and by how much?

(c) Index nested loop, `sessions` outer. Cost? Compare with (b) and explain the surprise.

(d) Hash join. Does the build side fit in $M = 42$? Cost. Then: how much `work_mem`, in MB, would make it fit, and what is the cost then?

(e) Sort-merge join: cost of sorting each input with $M = 42$, plus the merge. Total.

(f) Rank the five. Then add `WHERE s.id < 100` (100 sessions on 2 pages) and recompute the index nested loop, the block nested loop and the hash join. Which wins now, and what does this say about keeping all three algorithms?

**Problem: C3. Reading a plan** [10 pts]

```
Sort  (actual time=510.2..512.0 rows=18 loops=1)
  Sort Key: (sum(c.amount)) DESC
  Sort Method: quicksort  Memory: 26kB
  ->  HashAggregate  (actual time=509.8..510.0 rows=18 loops=1)
        Group Key: s.country
```

```
        Batches: 1  Memory Usage: 40kB
        ->  Hash Join  (actual time=61.4..452.7 rows=4000000 loops=1)
              Hash Cond: (c.session_id = s.id)
              ->  Seq Scan on clicks c  (actual time=0.02..120.9 rows=4000000
    loops=1)
              ->  Hash  (actual time=61.2..61.2 rows=100000 loops=1)
                    Buckets: 131072  Batches: 4  Memory Usage: 4096kB
                    ->  Seq Scan on sessions s  (actual time=0.01..35.0 rows
    =100000 loops=1)
Execution Time: 513.1 ms
```

(a) List the operators in execution order and mark the pipeline breakers.
(b) Explain `Batches: 4` on the Hash node. What would you change to make it `Batches: 1`, and roughly by how much?
(c) Why are the two numbers on the Hash node equal (61.2..61.2), and why does the Hash Join's first row appear at 61.4 ms?
(d) Why is the aggregate a `HashAggregate` and not a `Sort` + `GroupAggregate`? Use the row count.
(e) You run `SET enable_hashjoin = off` and `EXPLAIN ANALYZE` again. Sketch the plan you expect (which join, what feeds each side, what new line appears) and say whether it will be faster or slower, and why.
(f) Every node shows `loops=1`. Which plan shape would show `loops=100000` on some node, and what would that node be?

## Problem: C4. Duplicates and groups [6 pts]

(a) `SELECT DISTINCT user_id FROM clicks;` on 4,000,000 rows. Give the two ways to execute it. Which one does the planner choose when there are 50 distinct users, and which when there are 3,000,000? Why?
(b) `SELECT session_id, count(*) FROM clicks GROUP BY session_id;` where `clicks` has a B+ tree on `session_id`. Which aggregate does the planner use, is it a pipeline breaker, and how much memory does it hold at once?
(c) Explain in one paragraph why a sort-based aggregate keeps one group in memory while a hash-based one keeps all of them, and what that implies when the number of groups is near the number of rows.

## Problem: C5. Discussion [9 pts]

(a) Show with $B = 30$ pages that going from $M = 5$ to $M = 6$ cuts the sort I/O by a third. Then explain in words why one extra buffer can matter that much. [3 pts]
(b) "Hash join reads each input exactly once, so it is always the cheapest join." Give three distinct objections, each with a concrete situation. [3 pts]
(c) Compare nested-loop, sort-merge and hash join: for each, one situation where it is clearly the best choice and one where it is a disaster. Then say why a database keeps all three. [3 pts]